Report Lab 2

Kenan Augsburger

11 November 2025

Contents

1. Code analysis

The code analysis is done in the code in comments. If the code has been modified, the original comments can be found in the section addressing the issues or in some cases, the function is suffixed with `_old` and decorated with `@deprecated`

1.1. Global constants

At the start of the scripts, we have multiple constants defined which are the parameters for the secp256k1

```python
# --- secp256k1 parameters ---
P = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEFFFFFC2F
A = 0
B = 7
Gx
= 55066263022277343669578718895168534326250603453777594175500187360389116729240
Gy
= 32670510020758816978083085130507043184471273380659243275938904335757337482424
N = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141
```

When comparing with the parameters found online[1], we can see that those parameters are correct.

[1] https://neuromancer.sk/std/secg/secp256k1 °

2. Errors

This section covers each errors that were found.

> **Note**
>
> I forgot the part where the errors had to be sorted by criticality while writing this report.
>
> So here's the order of the vulnerabilities:
> 1. The usage of the `random` module instead of `secrets` (Section 2.1.2) since the key can be recovered.
> 2. The side channel attack on `scalar_mult` (Section 2.5) since it also leads to the key being recoverable but seems harder to pull off.
> 3. The insecure permission management of the private key (Section 2.3.3) since it once again leads to the private key leaking. In third place since this one requires the attacker to read files on the host. (Either any user access or with another service running on the host which allows path traversal or things like that.)
> 4. The domain issues (Section 2.4), the key isn't leaked but it's easy to get a signed record that's indistinguishable from a prescription.
> 5. The improper signature verification (Section 2.2) since it could be used to forge signatures but the requirements for this to be exploitable relies on the fact that whoever tries to validate a signature uses $\mathcal{O}$ as the public key
> 6. The bad range in `sign_message` (Section 2.1.1) since it will raise an exception. (Denial of service)
> 7. Destructive storage of private key (Section 2.3.2). Unlikely but if the public key isn't stored anywhere else (must be a trustworthy storage), anything signed with the original key becomes invalid.
> 8. Blind loading (Section 2.3.1). Also unlikely, leads to undefined behavior. (Mostly problematic when paired with the improper signature verification, but also if the private key isn't in $[1; N[$)

2.1. `random.randrange`

This function is used twice in the code.

```python
def generate_keypair() -> tuple[int, tuple[int, int]]:
    """Generate a pair of keys"""
    priv = random.randrange(1, N)
    pub = scalar_mult(priv, (Gx, Gy))
    return priv, pub
```

```python
def sign_message(priv, message):
    z = sha256(message)
    while True:
        k = random.randrange(0, N)
        R = scalar_mult(k, (Gx, Gy))
```

```
6           r = R[0] % N
7           if r == 0:
8               continue
9           k_inv = inv_mod(k, N)
10          s = (k_inv * (z + r * priv)) % N
11          if s == 0:
12              continue
13          return (r, s)
```

The documentation[2] for the module and its fonction starts with a warning to not use it for security and cryptographic purposes. This kind of warning shouldn't be ignored since it means that the prngs it provides are either not proved to be cryptographically secure or worse, proved to be attackable.

Furthermore, the range can be a problem independently of how good the distribution is. When multiplying a point with a scalar value, if the value is equal to zero or a multiple of N the result will be $\mathcal{O}$ which can be problematic.

For the first usage, the range gives us a value priv $\in \{1, ..., N-1\}$. This means that we shouldn't get a `None` but just to be safe we should have an assertion before returning our keys.

For the second usage, we have $k \in \{0, ..., N-1\}$ which means that R can be `None` which will raise an exception when accessing `R[0]`. This means that that there is a $\frac{1}{N}$ chance of having an issue when trying to sign a message.

We have two problems here.

2.1.1. Bad range $\{0, ..., N-1\}$

The problem here is that there is a chance of raising an exception and the implications vary depending on how exceptions are handled by the caller as well as which code paths lead to this function being executed.

The caller shouldn't expect `sign_message` to fail when given valid parameters. Depending on how they handle exceptions, the program could end up revealing informations which could be exploited or end up in an unrecoverable state.

If an attacker can trigger this function a lot, it could be used to leak secrets or for a denial of service attack.

If the exception results in an unrecoverable state such as the program exiting There is a serious risk of service degradation proportional to the usage.

In simple terms, the risk is that the service becomes unavailable or worse, that secrets could be leaked and used by bad actors to falsify prescriptions.

The simple fix is to change the range to $\{1, ..., N-1\}$. (Using an assertion as a sanity check and also allows the typing system to narrow after the multiplication)

[2] https://docs.python.org/3/library/random.html°

```python
def sign_message(priv, message):
    z = sha256(message)
    while True:
        k = random.randrange(1, N)
        R = scalar_mult(k, (Gx, Gy))
        assert R
        r = R[0] % N
        if r == 0:
            continue
        k_inv = inv_mod(k, N)
        s = (k_inv * (z + r * priv)) % N
        if s == 0:
            continue
        return (r, s)
```

Python

As stated previously, the range used in `generate_keypair` is correct but we should still have an assertion after the multiplication since it doesn't cost us much and having $\mathcal{O}$ as a public key would be catastrophic. (since $\forall k \in \mathbb{Z}, k\mathcal{O} = \mathcal{O}$)

```python
def generate_keypair() -> tuple[int, Point]:
    priv = random.randrange(1, N)
    pub = scalar_mult(priv, (Gx, Gy))
    assert pub
    return priv, pub
```

Python

2.1.2. Usage of `random`

```python
# WARN: the secrets module should be used instead
#   https://docs.python.org/3/library/random.html
import random

# INFO: Dangerous use of random, otherwise ok
def generate_keypair() -> tuple[PrivKey, PubKey]:
    """Generate a pair of keys"""
    # Take a random value between 1 and N (N not included)
    # WARN: not cryptographically secure
    priv = random.randrange(1, N)
    # ...

# INFO: issue with random but otherwise valid ECDSA
def sign_message(priv: PrivKey, message: Buffer) -> Signature:
    """Signs a message using ECDSA with sha256 as the hash function"""
    z = sha256(message)
    while True:
        # WARN: not cryptographically secure + 0 included
        # HACK: k = random.randrange(0, N)
```

Python

```
20          k = random.randrange(1, N)
21          # ...
```

The `random` module clearly warns against its use for security or cryptographic purposes. This is a PRNG module and not a CSPRNG. I don't have an exact exploit for our scenario but looking online, there are multiple articles going in depth on how the `random` module can be broken.[3][4]

If an attacker find the internal state or the seed of the random number generator, they can use it to generate the same private key. If they get the private key, it's game over and they can create their own records with valid signatures.

We need to fix this to prevent bad actors from having any way of recovering the private key.

As stated by the documentation of `random`, the secrets module should be used instead.

```python
import secrets

def generate_keypair() -> tuple[PrivKey, PubKey]:
    """Generate a pair of keys"""
    # Take a random value between 1 and N (N not included)
    priv = secrets.randbelow(N - 1) + 1
    # ...

def sign_message(priv: PrivKey, message: Buffer) -> Signature:
    """Signs a message using ECDSA with sha256 as the hash function"""
    z = sha256(message)
    while True:
        k = secrets.randbelow(N - 1) + 1
        # ...
```

2.2. Improper signature verification

> **Note**
>
> The `verify_signature` function is never called. This probably isn't an issue since this is supposed to be a library and we don't have a function that reads a record from the disk.

```python
def verify_signature(pub: PubKey, message: Buffer, signature:
Signature) -> bool:
    """Check the ECDSA signature"""
    # WARN: Missing the first step of verification.
    #   pub should be a valid public key (on the curve and not point at
    infinity)
    r, s = signature
    # r and s are expected to be in NN^* and mod n
```

[3] https://bitsdeep.com/projects/python-random-prediction/
[4] https://stackered.com/blog/python-random-prediction/

```
    if not (1 ⩽ r < N and 1 ⩽ s < N):
        return False
    # compute z = H(M)
    z = sha256(message)
    # s^(-1) mod n
    s_inv = inv_mod(s, N)
    # u1 = H(M)/s (mod n)
    u1 = (z * s_inv) % N
    # u2 = r/s (mod n)
    u2 = (r * s_inv) % N
    P1 = scalar_mult(u1, (Gx, Gy))
    P2 = scalar_mult(u2, pub)
    # P = (x1, y1) = u1G + u2A
    P = point_add(P1, P2)
    if P is None:
        return False
    # check if r = x1 mod n
    return (P[0] % N) == r
```

The signature verification doesn't comply with what we've seen in class[5]. Specifically, the first step is skipped which is to check that $A \neq \mathcal{O}$ and that A is on the curve

This means that if the public key $\mathcal{O}$ is given to check a signature, we have the following:

$$\begin{aligned} u_1 &= \frac{H(M)}{s} && \pmod n \\ u_2 &= \frac{r}{s} && \pmod n \\ (x_1, y_1) &= u_1 G + u_2 A \\ &= u_1 G + u_2 \mathcal{O} \\ &= u_1 G \\ &= \frac{H(M)}{s} G && \pmod n \\ r &\stackrel{?}{=} x_1 && \pmod n \end{aligned} \tag{1}$$

Since there is no check for $A = \mathcal{O}$, we can simply compute with a chosen s and M

$$(x_1, y_1) = \frac{H(M)}{s} G \tag{2}$$

And extract x_1 to use as our value for r. The following snippet uses $s = 1$ which means that we have

$$(r, s) = \left(\left(\frac{H(M)}{1} G \right)_x, 1 \right) \tag{3}$$

[5] https://cyberlearn.hes-so.ch/pluginfile.php/4064747/mod_resource/content/0/03_asymmetric.pdf#page=47°

```python
def forge_signature_for_infinity_pub(message: Buffer) -> Point:
    """Generate a valid signature when public key is None"""
    z = sha256(message)
    s = 1
    s_inv = inv_mod(s, N)
    u1 = (z * s_inv) % N
    point = scalar_mult(u1, (Gx, Gy))
    assert point is not None
    return (point[0] % N, s)

forged = forge_signature_for_infinity_pub(b"test message")
if verify_signature(None, b"test message", forged):
    print("forged successfully")
```

If the value `None` is given as the public key, an attacker can easily forge a signature. Thus, this should be fixed to preserve the authenticity.

The fix is simple, just check if `pub` is `None`

```python
def verify_signature(pub: PubKey, message: Buffer, signature: Signature) -> bool:
    """Check the ECDSA signature"""
    if pub is None or not is_on_curve(pub):
        return False
    r, s = signature
    # r and s are expected to be in NN^* and mod n
    if not (1 <= r < N and 1 <= s < N):
        return False
    # compute z = H(M)
    z = sha256(message)
    # s^(-1) mod n
    s_inv = inv_mod(s, N)
    # u1 = H(M)/s (mod n)
    u1 = (z * s_inv) % N
    # u2 = r/s (mod n)
    u2 = (r * s_inv) % N
    P1 = scalar_mult(u1, (Gx, Gy))
    P2 = scalar_mult(u2, pub)
    # P = (x1, y1) = u1G + u2A
    P = point_add(P1, P2)
    if P is None:
        return False
    # check if r = x1 mod n
    return (P[0] % N) == r
```

> **Note**
>
> This issue was hinted by Mario, I didn't notice it at first because I knew that `generate_keypair` couldn't give $\mathcal{O}$ as the public key unless the private key is a multiple of N or equal to 0.

2.3. `load_or_generate_keys`

2.3.1. Blind loading

```python
if os.path.exists(priv_file) and os.path.exists(pub_file):
    # If both files exist, proceed to extract the private and public keys.
    with open(priv_file, "r") as f:
        # The private key is converted from a base16 string to an int.
        # NOTE: The fact that we strip what we read shouldn't matter unless
        # the key isn't stored properly
        priv = int(f.read().strip(), 16)
    with open(pub_file, "r") as f:
        # The public key is read as a JSON object with the keys x and y
        # containing integers in base64
        pub_data = json.load(f)
        pub = (int(pub_data["x"], 16), int(pub_data["y"], 16))
    print("Loaded existing keypair from disk.")
    # WARN: if both files exist, their value is taken as ground truth. There
    #   is no verification that the private key is in [1;N[ and that the
    #   public key is the result of priv * G
```

When loading the pair of keys from disk, there are no checks made to validate it. The problem is that to be able to properly sign and verify the signature of a message you need the private key a to respect $1 \leq a < N$ and the public key A should be $A = aG$.

If $a = 0$, the corresponding public key $A = \mathcal{O}$ which would be unusable and raise an exception whenever accessing either the x or y components of the point.

If $a = N$, then $A = \mathcal{O}$ since N is the order of the group. Same consequences.

Any other value of a will be treated as $a \bmod N$ so as long as $a \bmod N \neq 0$ the key should be valid but there shouldn't be any reason why we should have longer keys since it won't provide any benefits.

If $A \neq aG$, the signature verification won't work.

If A isn't on the curve, the program will stop after an assertion in `scalar_mult`.

If an attacker can modify the keys then you've got bigger problems than the program crashing from invalid keys.

One way this could go wrong is if there is a partial recovery, or maybe one of the keys was missing and the remaining key wasn't writable during the creation of a new pair. In those instances, the pair wouldn't match and lead to the program crashing.

This should be fixed because an invalid pair of keys will make the program unusable.

The simple fix would be to check that the keys are valid right after loading them and raise an exception to be handled otherwise. To make it easier to handle down the line, it also makes sense to split `load_or_generate_keys` into two distinct functions

```python
def compute_public_key(priv: PrivKey) -> PubKey:
    """Compute the public key of a private key"""
    pub = scalar_mult(priv, (Gx, Gy))
    assert pub is not None
    return pub


def is_valid_private_key(key: PrivKey) -> bool:
    """Check if a private key is valid with the curve"""
    return 1 <= key < N


def is_valid_public_key(pub: PubKey, priv: PrivKey | None) -> bool:
    """Check if a public key is valid
    If None is specified for the private key, this function only checks
    if the
    key is on the curve
    """
    return is_on_curve(pub) if priv is None else pub ==
    compute_public_key(priv)


class InvalidPrivateKeyError(Exception):
    pass


class InvalidPublicKeyError(Exception):
    pass


def load_priv_key(path: str) -> PrivKey:
    """Load a private key from disk and check its validity
    Raises:
        FileNotFoundError: If the path doesn't lead to an existing file
        InvalidPrivateKeyError: If the key is invalid
    """
    if not os.path.exists(path):
        raise FileNotFoundError()
    with open(path, "r") as f:
        priv = int(f.read().strip(), 16)
        if not is_valid_private_key(priv):
            raise InvalidPrivateKeyError()
```

```
41          return priv
42
43
44  def load_pub_key(path: str, priv_key: PrivKey | None) -> PubKey:
45      """Load a public key from the disk and check its validity.
46      Raises:
47          FileNotFoundError: If the path doesn't lead to an existing file
48          InvalidPublicKeyError: If the public key isn't the public key of the
49              provided private key. If None is given for the private key,
                The only
50              check is whether the public key is a point on the curve.
51      """
52      if not os.path.exists(path):
53          raise FileNotFoundError()
54      with open(path, "r") as f:
55          # The public key is read as a JSON object with the keys x and y
56          # containing integers in base64
57          pub_data = json.load(f)
58          pub = (int(pub_data["x"], 16), int(pub_data["y"], 16))
59          if not is_valid_public_key(pub, priv_key):
60              raise InvalidPublicKeyError()
61          return pub
62
63
64  def generate_and_save_keys(priv_path: str, pub_path: str) -> tuple[PrivKey,
    PubKey]:
65      if os.path.exists(priv_path):
66          # NOTE: only keeping one backup since I don't want to bother with
            numbering
67          os.replace(priv_path, priv_path + ".bak")
68          print(
69              f"existing private key found during generation of keys, backing
                up to {priv_path}.bak"
70          )
71
72      if os.path.exists(pub_path):
73          # NOTE: only keeping one backup since I don't want to bother with
            numbering
74          os.replace(pub_path, pub_path + ".bak")
75          print(
76              f"existing public key found during generation of keys, backing up
                to {pub_path}.bak"
77          )
78
79      priv, pub = generate_keypair()
80      # WARN: The public and private keys are both simply saved to disk
81      #   without any specifications when it comes to permissions. (on linux,
82      #   resulted in 644 permissions)
```

```
    with open(priv_path, "w") as f:
        # The private key is simply written properly in hex format
        f.write(hex(priv))
    with open(pub_path, "w") as f:
        # The public key is saved correctly in JSON
        json.dump({"x": hex(pub[0]), "y": hex(pub[1])}, f)
    print("New keypair generated and saved.")
    return (priv, pub)


def generate_and_save_public_key(priv: PrivKey, pub_path: str) -> PubKey:
    if os.path.exists(pub_path):
        # NOTE: only keeping one backup since I don't want to bother with
        numbering
        os.replace(pub_path, pub_path + ".bak")
        print(
            f"existing public key found during generation of keys, backing up
            to {pub_path}.bak"
        )

    pub = compute_public_key(priv)
    with open(pub_path, "w") as f:
        # The public key is saved correctly in JSON
        json.dump({"x": hex(pub[0]), "y": hex(pub[1])}, f)
    return pub


def load_or_generate_keys() -> tuple[PrivKey, PubKey]:
    """This function loads an existing pair of private/public keys or
    generates,
    saves and return a new pair.
    """

    # Files containing the private and public keys
    priv_file = "ecdsa_private.key"
    pub_file = "ecdsa_public.key"

    try:
        priv = load_priv_key(priv_file)
    except (InvalidPrivateKeyError, FileNotFoundError) as err:
        print(
            "Your private key is invalid."
            if isinstance(err, InvalidPrivateKeyError)
            else "Missing private key"
        )
        choice = input(
            "Would you like to generate a new pair of keys? Existing keys
            will be backed up to [key].bak y/N: "
```

```
127         )
128         if not choice == "y":
129             raise err
130         return generate_and_save_keys(priv_file, pub_file)
131
132     try:
133         pub = load_pub_key(pub_file, priv)
134     except (InvalidPublicKeyError, FileNotFoundError) as err:
135         print(
136             "Your public key is invalid."
137             if isinstance(err, InvalidPublicKeyError)
138             else "Missing public key"
139         )
140         choice = input("Would you like to generate the public key? y/N: ")
141         if not choice == "y":
142             raise err
143         return (priv, generate_and_save_public_key(priv, pub_file))
144
145     return (priv, pub)
```

> **Note**
>
> This rewrite of the `load_or_generate_keys` is a bit messy but it handles the issue of loading invalid keys as well as the issue of irreversibly losing keys without warning. (Described in more details in the next section)

2.3.2. Destructive storage of private key

```python
else:
    print("No keypair found — generating new one...")
    # If any of the files do not exist we generate a new pair of keys
    priv, pub = generate_keypair()
    with open(priv_file, "w") as f:
        # The private key is simply written properly in hex format
        f.write(hex(priv))
```

If only private key exists, the function will overwrite it with a new private key. This is destructive since we can always compute the public key from the private key but we cannot recover the private key once overwritten.

This means that if for some reasons, the public key isn't found we will get a new pair of keys. We will need to distribute our new public key to anyone who needs to verify our signatures and if we can't our old public key anywhere, the validity of past records becomes unverifiable.

This most likely isn't an attack vector. If an attacker manages to delete the public key on the server they probably already compromised the server.

This should be fixed because it's a threat to the trust of previously signed records.

The simple fix to this issue is to prompt the user and ask whether they want to overwrite the private key or compute the public key and save it.

This is implemented in the fix of the previous section.

2.3.3. Insecure permission management of private key

```python
else:
    print("No keypair found — generating new one...")
    # If any of the files do not exist we generate a new pair of keys
    priv, pub = generate_keypair()
    with open(priv_file, "w") as f:
        # The private key is simply written properly in hex format
        f.write(hex(priv))
    with open(pub_file, "w") as f:
        # The public key is saved correctly in JSON
        json.dump({"x": hex(pub[0]), "y": hex(pub[1])}, f)
    print("New keypair generated and saved.")
    # WARN: The public and private keys are both simply saved to disk
    #   without any specifications when it comes to permissions. (on linux,
    #   resulted in 644 permissions)
```

When either a public key or private key file is missing, a new pair of keys is generated and saved to disk using the built-in open function in `'w'` mode.

The keys will have the default permissions (determined by the OS). On Linux, it often means `0644` which is an issue since we don't want any other user to be able to read the private key.

In the event of an attacker gaining access to the host running the program, they can extract the private key without any privilege escalation. This would allow them to falsify records. (Breaks authenticity and integrity)

Effectively, this would mean that the attacker could create a fake prescription and the pharmacy would treat it as valid. For doctors, this would allow the attacker to create fake symptoms for a patient which in turn would result in more work for the doctors to check and effectively destroy the efficiency brought by the system.

This issue should be fixed because it could cause serious legal troubles since it involves medical prescriptions. Furthermore, it also risks the loss of trust in the system.

```python
priv, pub = generate_keypair()
(priv_mask, pub_mask) = (0o600, 0o644)
flags = os.O_WRONLY | os.O_CREAT | os.O_TRUNC

# Get open the file through the os api to enforce permissions on the files
priv_fd = os.open(priv_path, flags, priv_mask)
os.fchmod(priv_fd, priv_mask)

with os.fdopen(priv_fd, "w") as f:
    # The private key is simply written properly in hex format
```

```
    f.write(hex(priv))

pub_fd = os.open(pub_path, flags, pub_mask)
os.fchmod(pub_fd, pub_mask)
with os.fdopen(pub_fd, "w") as f:
    # The public key is saved correctly in JSON
    json.dump({"x": hex(pub[0]), "y": hex(pub[1])}, f)
print("New keypair generated and saved.")
```

2.4. Domain issues

This issue involves the design of the program and isn't specific to a snippet or two of code. This is mainly about what gets signed, how it's structured and which keys are used.

```python
def build_doctor_record() -> bytes:
    """Build a json array with the following structure
    [
        name,
        avs,
        { [drug]: dosage }
    ]
    """
    name = input("Patient name: ").strip()
    avs = input("Patient AVS number: ").strip()
    drugs = {}
    print("Enter drugs and dosages. Leave drug name empty to finish.")
    while True:
        drug = input("Drug name: ").strip()
        if not drug:
            break
        dosage = input("Dosage (string): ").strip()
        drugs[drug] = dosage
    return json.dumps([name, avs, drugs]).encode("utf-8")


def build_patient_record() -> bytes:
    """Build a json array with the following structure
    [
        name,
        avs,
        { [symptom]: temporality }
    ]
    """
    name = input("Patient name: ").strip()
    avs = input("Patient AVS number: ").strip()
    symptoms = {}
    print("Enter symptoms and their temporality. Leave symptom name empty
    to finish.")
```

```
    while True:
        sym = input("Symptom: ").strip()
        if not sym:
            break
        temp = input("Temporality (string): ").strip()
        symptoms[sym] = temp
    return json.dumps([name, avs, symptoms]).encode("utf-8")
```

Both of these functions give us JSON with the following type:

```typescript
type Record = [
  string,
  string,
  {
    [key: string]: string
  }
]
```

Before saving, the only way to distinguish between the two types of records is the flag `isPatient`. Once saved to disk, the distinction is whether it comes from `medical_records.txt` or `prescriptions_records.txt`. (The latter isn't that big of an issue since it would be the same if we saved to a database using different tables.) The issue is that if the pharmacy is given patient record with symptoms such as `paracetamol` or `concerta` and the corresponding signature they aren't able to tell that it cannot be used as a prescription.

Furthermore, when looking at what's signed and what's saved, we have another issue.

```python
def main():
    #...
    signature = sign_message(priv, record)
    save_record_to_db(record, signature, isPatient)

def save_record_to_db(record: bytes, signature: Signature, isPatient: bool):
    """Save a patient record to the database accompanied by its signature"""
    # WARN: There is no guarantee that given signature is for the given record
    #   but this shouldn't be an issue when it comes to saving
    # WARN: The timestamps isn't part of the signature, which means that it
    #   shouldn't be interpreted as valid because the signature is
    entry = {
        "timestamp": datetime.now().isoformat(),
        "record": record.decode(),
        "signature": {"r": hex(signature[0]), "s": hex(signature[1])},
    }
    # Choose which file to use based on the isPatient flag
    filename = "medical_records.txt" if isPatient else
    "prescriptions_records.txt"
    # WARN: The entry is appended to the file. This will result in an invalid
```

```
    # json structure which will need to be handled correctly when reading
    from it
    with open(filename, "a", encoding="utf-8") as f:
        f.write(json.dumps(entry, ensure_ascii=False) + "\n")
```

The attached timestamp isn't part of the signature. This means that changing the timestamp wouldn't invalidate a prescription. You could reuse a prescription and simply change the date.

Also related to the idea of having a timestamp, we need to rotate the private key periodically. This rotation shouldn't invalidate previous signatures but currently there is no way to know which public key should be used to validate a signature.

Here's a list of what an attacker could do depending on what they have access to:

- If an attacker can force or accelerate a key rotation, they can invalidate all the previous signatures.
- If an attacker can create a patient record, get it signed and use it at a pharmacy as if it's a prescription it would pass as valid.
- If an attacker had a valid prescription in the past and are able to manipulate the timestamp, they can fake a new identical prescription.

Without an attacker, the key rotation will still invalidate previous signatures.

Once again, this should be fixed because it's a liability issue if prescriptions can be forged using our system or if patients are unable to use their prescriptions after a key rotation.

The first fix is to simply include the timestamp and type of record in the signature

```python
T = TypeVar("T")


@dataclass(frozen=True)
class Record(Generic[T]):
    """Class representing a value that should be serializable and
    deserializable
    and include a timestamp that match its creation.
    """

    data: T
    timestamp: datetime = datetime.now(timezone.utc)
    pass

    @property
    def type(self) -> str:
        return self.data.__class__.__name__

    def to_dict(self) -> dict:
        return {
            "type": self.type,
            "data": self.data,
```

```
            "timestamp": self.timestamp.isoformat(),
        }


class DoctorData(NamedTuple):
    name: str
    avs: str
    drugs: dict[str, str]


class PatientData(NamedTuple):
    name: str
    avs: str
    symptoms: dict[str, str]


def build_doctor_record() -> Record[DoctorData]:
    """Build a doctor record"""
    name = input("Patient name: ").strip()
    avs = input("Patient AVS number: ").strip()
    drugs: dict[str, str] = {}
    print("Enter drugs and dosages. Leave drug name empty to finish.")
    while True:
        drug = input("Drug name: ").strip()
        if not drug:
            break
        dosage = input("Dosage (string): ").strip()
        drugs[drug] = dosage
    return Record(data=DoctorData(name, avs, drugs))

def build_patient_record() -> Record[PatientData]:
    """Build a patient record"""
    name = input("Patient name: ").strip()
    avs = input("Patient AVS number: ").strip()
    symptoms: dict[str, str] = {}
    print("Enter symptoms and their temporality. Leave symptom name empty
    to finish.")
    while True:
        sym = input("Symptom: ").strip()
        if not sym:
            break
        temp = input("Temporality (string): ").strip()
        symptoms[sym] = temp
    return Record(data=PatientData(name, avs, symptoms))

def save_record_to_db(record: Record, priv: PrivKey):
    """Save a patient record to the database accompanied by its signature"""
    json_record = json.dumps(record.to_dict())
```

```
69      signature = sign_message(priv, json_record.encode("utf-8"))
70      entry = {
71          "record": json_record,
72          "signature": {"r": hex(signature[0]), "s": hex(signature[1])},
73      }
74      filename: str
75      match record.data:
76          case PatientData():
77              filename = "medical_records.txt"
78          case DoctorData():
79              filename = "prescriptions_records.txt"
80          case _:
81              raise ValueError(f"Unexpected record received
                Record[{record.type}]")
82      # WARN: The entry is appended to the file. This will result in an invalid
83      # json structure which will need to be handled correctly when reading
        from it
84      with open(filename, "a", encoding="utf-8") as f:
85          f.write(json.dumps(entry, ensure_ascii=False) + "\n")
86
87
88  def main():
89      # ...
90      choice = input("Choose 1 or 2: ").strip()
91      if choice == "1":
92          record = build_doctor_record()
93      elif choice == "2":
94          record = build_patient_record()
95          # NOTE: redundant
96      else:
97          print("Invalid choice")
98          return
99      save_record_to_db(record, priv)
```

This approach fixes the issue of the type of record being lost by including it in the type and serialization. The type and timestamps are both included in the signature.

> **Note**
>
> This doesn't address the issue of key rotation but helps. Since the timestamp is included in the signature, we need a versioning system for the public keys and the function that would deserialize the record should check the timestamp, find the closest public key before that timestamp.
>
> This isn't implemented in the fix because the original program doesn't implement anything past saving to disk and doing so would take more time without being too useful for this lab.
>
> (The versioning of public keys would also be complex to do right. We probably need a master key to sign the public keys and make sure that those keys are valid when loading the program)

> **Warning**
>
> Looking back, the fix introduces another issue related to the timestamp. The timestamp is set when the Record is instanciated and not when it's signed. Rather than having a `Record` class to serialize then sign, the `Record` class shouldn't have the timestamp but instead provide a `sign` method to produce a read only `SignedRecord` which would add a timestamp, serialize and sign on instantiation.

2.5. Side channel attack

> **Note**
>
> This issue was mentioned by Mario.

The following snippet is mathematically correct and implements the double and add[6] algorithm for elliptic curve point multiplication.

Python

```python
# INFO: mathematically sound, None means point to infinity
def scalar_mult(k: int, point: Point | None) -> Point | None:
    """Multiply a point on the curve by the scalar value k"""
    # Check that the point is on the curve otherwise the multiplication isn't
    # possible.
    assert is_on_curve(point)
    if k % N == 0 or point is None:
        # If k is perfectly divisible by N or now point was given, the result
        is None.
        # NOTE: If k is perfectly divisible by N, the result will be a None,
        # this check saves time
        return None
```

[6] https://en.wikipedia.org/wiki/Elliptic_curve_point_multiplication#Double-and-add

```
12      if k < 0:
13          # If k is negative, we simply multiply both inputs by -1 which is
14          # simple to do and allows the next part of the algorithm to work.
15          return scalar_mult(-k, (point[0], (-point[1]) % P))
16      result = None
17      addend = point
18      # To calculate the result, we add point k times, effectively multiplying
19      # the point by k
20      while k:
21          if k & 1:
22              result = point_add(result, addend)
23          addend = point_add(addend, addend)
24          k >>= 1
25      return result
```

It was mentioned multiple times during class that this algorithm is vulnerable to timing attacks. The compute time scales with the number for bits set to 1 which allows an attacker to guess the scalar value more efficiently.

This means that the following functions are vulnerable to timing attacks since they use double and add.

- `generate_keypair` : if the attacker gets info on the private key and know the public key, they can find the private key (probably still leaves a lot of bruteforce but at a smaller scale).
- `sign_message` : if the attacker manages to find k, they can compute the private key.
- `verify_signature` : this one shouldn't be an issue since it doesn't involve the private key.

The reason why we should fix this is the same as every other issue which leaks the private key. This would allow an attacker to forge signatures and thus break authenticity and integrity.

To fix this, we need an algorithm that's resistant to side channel attacks[7]. The main recommandation when looking online is to use the Montgomery ladder.

When looking online[8], the `secp256k1` curve doesn't support Montgomery ladder. We would need to change the curve in order to use it.

One fix would be to use a curve that supports the Montgomery ladder and use the Montgomery curve form since the algorithm is optimized for those curves. We could use EC25519[9] since it's widely used and is a Montgomery curve.

We would then need to update the `point_add` function to work with the new curve and implement the Montgomery ladder for `scalar_mult` .

[7] https://eitca.org/cybersecurity/eitc-is-acc-advanced-classical-cryptography/elliptic-curve-cryptography/elliptic-curve-cryptography-ecc/examination-review-elliptic-curve-cryptography-ecc/how-does-the-double-and-add-algorithm-optimize-the-computation-of-scalar-multiplication-on-an-elliptic-curve/°

[8] https://safecurves.cr.yp.to/ladder.html°

[9] https://en.wikipedia.org/wiki/Curve25519°

> **Note**
>
> Since this fix seems to take quite a bit of time, I'm skipping it for now.

3. Conclusion

Other than cryptographic issues, there are also a lot of problems with the structure of the code. If the goal is to provide a library to use within another program, it should be clearly structured in modules and ensure that the library is used correctly.

We would need classes to ensure that public and private keys are valid on instantiation as well as clear exceptions to handle errors.

4. AI usage

I used AI while writing the fixes for syntax references, mainly how to use the typing module. Here are some of the prompts used.

> **Prompt**
>
> how can I mark a function as deprecated in a way that is picked up by the lsp?

> **Prompt**
>
> what are the standard exceptions in python?

> **Prompt**
>
> How can I write to a file in python and set its permission before any data is written?
>
> > **Note**
> >
> > When looking online, the solution was always to use `os.chmod` but I didn't like that solution since it only worked if the file already existed.

> **Prompt**
>
> How can I properly type a dict?